Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also compare across forks or learn more about diff comparisons.

base: main ← compare: c0c25029/アイテム機能

✓ **Able to merge.** These branches can be automatically merged.

Discuss and review the changes in this comparison with others. Learn about pull requests **Create pull request**

1 commit | 1 file changed | 1 contributor

Commits on Nov 3, 2026

アイテム機能

c0c25029dd committed 11 minutes ago d1c01dc

Showing 1 changed file with **77 additions** and **1 deletion**. Split Unified

78 geme_base.py

```
19  19    COLOR_ENEMY = (255, 50, 50)    # 赤色（敵）
20  20    COLOR_WHITE = (255, 255, 255)  # 白色
21  21    COLOR_RED = (200, 0, 0)        # 強調用赤
    22  + COLOR_HEAL = (0, 255, 0)       # 回復アイテム
    23  + COLOR_ITEM_SCORE = (255, 255, 0)  # スコアアイテム
22  24
23  25
24  26    class Player:
98  100                   self.hp = 0
99  101               self.invincible_timer = 60  # 1秒間（60フレーム）の無敵時間
100 102
    103 +     def heal(self, amount: int) -> None:
    104 +         """HPを回復する（最大HPを超えない）"""
    105 +         self.hp = min(self.max_hp, self.hp + amount)
    106 +
101 107
102 108   class Enemy:
103 109       """敵キャラクターの移動と描画を管理するクラス"""
132 138           return (self.x, self.y)
133 139
134 140
    141 + class Item:
    142 +     """アイテムの種類、位置、描画を管理するクラス"""
    143 +
```

```
+    def __init__(self, x: float, y: float, item_type: str) -> None:
+        self.x: float = x
+        self.y: float = y
+        self.radius: int = 10
+        self.item_type: str = item_type
+
+    def draw(self, screen: pygame.Surface) -> None:
+        """アイテムを種類に応じた色で描画する"""
+        if self.item_type == "HEAL":
+            color = COLOR_HEAL
+        else:
+            color = COLOR_ITEM_SCORE
+
+        pygame.draw.circle(
+            screen, color,
+            (int(self.x), int(self.y)), self.radius
+        )
+
+    def get_pos(self) -> tuple[float, float]:
+        """アイテムの座標を返す"""
+        return (self.x, self.y)
+
+
class Game:
    """ゲーム全体の状態、時間、スコア、当たり判定を管理するクラス"""

        """ゲームの状態を初期化またはリスタートする"""
        self.player: Player = Player(float(SCREEN_WIDTH / 2), float(SCREEN_HEIGHT / 2))
        self.enemies: list[Enemy] = []
+       self.items: list[Item] = []
+       self.item_spawn_timer: int = 0
+       self.item_score: int = 0
        self.elapsed_frames: int = 0
        self.spawn_timer: int = 0
        self.game_state: str = "PLAYING"  # "PLAYING", "GAME_OVER", "CLEAR"
                # プレイヤーの更新
                self.player.update(keys)

+               # アイテムの出現処理（約10秒ごと）
+               self.item_spawn_timer += 1
+               if self.item_spawn_timer >= 600:
+                   self.item_spawn_timer = 0
+                   self.spawn_item()
+
                # 時間経過のカウント
                self.elapsed_frames += 1
                survived_seconds = self.elapsed_frames // FPS
                for enemy in self.enemies:
                    enemy.update(px, py)

+               # アイテムの取得判定
+               self.check_item_collisions()
+
                # 当たり判定のチェック
                self.check_collisions()
```

```
227 271             self.enemies.append(Enemy(float(x), float(y)))
228 272
    273 +     def spawn_item(self) -> None:
    274 +         """フィールド上にランダムなアイテムを生成する"""
    275 +         x = random.randint(30, SCREEN_WIDTH - 30)
    276 +         y = random.randint(50, SCREEN_HEIGHT - 30)
    277 +
    278 +         item_type = random.choice(["HEAL", "SCORE"])
    279 +         self.items.append(Item(float(x), float(y), item_type))
    280 +
    281 +     def check_item_collisions(self) -> None:
    282 +         """プレイヤーとアイテムの接触判定・取得処理"""
    283 +         px, py = self.player.get_pos()
    284 +
    285 +         for item in self.items[:]:
    286 +             ix, iy = item.get_pos()
    287 +             distance = math.hypot(px - ix, py - iy)
    288 +
    289 +             if distance < self.player.radius + item.radius:
    290 +                 if item.item_type == "HEAL":
    291 +                     self.player.heal(30)
    292 +                 elif item.item_type == "SCORE":
    293 +                     self.item_score += 100
    294 +
    295 +                 self.items.remove(item)
    296 +
229 297         def check_collisions(self) -> None:
230 298             """プレイヤーとすべての敵の接触判定を行う"""
231 299             px, py = self.player.get_pos()
250 318             for enemy in self.enemies:
251 319                 enemy.draw(self.screen)
252 320
    321 +         # アイテム描画
    322 +         for item in self.items:
    323 +             item.draw(self.screen)
    324 +
253 325             # UI（HP、スコア、残り時間）の描画
254 326             hp_text = self.font.render(f"HP: {self.player.get_hp()}/{self.player.max_hp}",
    True, COLOR_WHITE)
255 327             self.screen.blit(hp_text, (20, 20))
256 328
257 329             survived_seconds = self.elapsed_frames // FPS
258     -           score_text = self.font.render(f"Score (Time): {survived_seconds}s", True,
    COLOR_WHITE)
    330 +         total_score = survived_seconds + self.item_score
    331 +
    332 +         score_text = self.font.render(
    333 +             f"Score: {total_score}", True, COLOR_WHITE
    334 +         )
259 335             self.screen.blit(score_text, (330, 20))
260 336
261 337             remaining_seconds = max(0, TIME_LIMIT - survived_seconds)
```